SQL Joins

Theory notes: concepts and types of joins

Covered in this document
1. What is a join and why it is needed
2. Keys and the join condition
3. INNER JOIN
4. LEFT (OUTER) JOIN
5. RIGHT (OUTER) JOIN
6. FULL OUTER JOIN
7. CROSS JOIN
8. Comparison, row-count behaviour and common pitfalls

1. What is a Join?

A relational database stores data in many separate tables rather than in one large table. This design, called **normalization**, reduces redundancy and keeps data consistent: a customer's details are stored once in a Customers table, and each order simply refers to that customer instead of repeating the details.

A **join** is the operation that combines rows from two (or more) tables into a single result set, based on a relationship between them. It lets us recombine the information that normalization split apart, so that a single query can answer questions such as which customer placed which order.

Conceptually, a join takes two tables, which we call the **left table** (A) and the **right table** (B), compares rows of A with rows of B using a **join condition**, and builds new rows from each pair that satisfies the condition. The different join types differ only in what they do with rows that have *no* matching partner.

2. Keys and the Join Condition

Joins rely on keys. A **primary key** uniquely identifies each row in a table. A **foreign key** is a column in one table that refers to the primary key of another table. The join condition normally compares a foreign key with the primary key it references, usually using equality (an *equi-join*).

Relationships between tables determine how many rows a join can produce:

- **One-to-one:** each row in A matches at most one row in B.
- **One-to-many:** one row in A can match many rows in B (for example, one customer and many orders). The 'one' side is repeated for every match.
- **Many-to-many:** rows on both sides can match many rows on the other side; this is normally resolved through a third, linking table.

2.1 The Sample Data Used for Explanation

All explanations below use the same two small tables so the behaviour of each join can be compared directly. Table A is **Customers**, and table B is **Orders**. The column *CustomerID* links them.

A: Customers

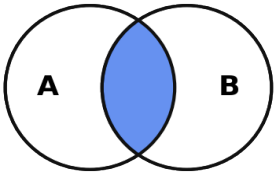
CustomerID	Name
1	Asha
2	Bala
3	Charu

B: Orders

OrderID	CustomerID	Amount
101	1	500
102	1	300
103	2	700
104	9	200

Notice two deliberate features of the data. Customer 3 (Charu) has **no orders**, and order 104 refers to CustomerID 9, which **does not exist** in Customers. These unmatched rows are exactly what distinguishes one join type from another.

3. INNER JOIN



Definition

An INNER JOIN returns only the rows for which the join condition is true in **both** tables. It is the intersection of the two tables with respect to the join condition. If INNER JOIN is written simply as JOIN, it means the same thing, because inner is the default join type.

How it works

For each row in A, the database looks for rows in B that satisfy the condition. Every matching pair produces one output row containing columns from both tables. Rows in A with no partner in B, and rows in B with no partner in A, are discarded.

When to use it

Use it when you are interested only in records that have a related record on the other side, for example, only customers who have actually placed orders. It is the most commonly used join.

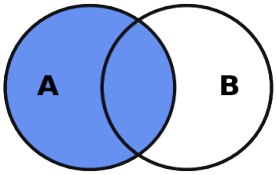
Handling of unmatched rows and NULLs

Unmatched rows are dropped completely, so no NULLs are produced by the join itself. Customer 3 (no orders) and order 104 (no customer) do not appear.

Result on the sample data (3 rows):

CustomerID	Name	OrderID	Amount
1	Asha	101	500
1	Asha	102	300
2	Bala	103	700

4. LEFT (OUTER) JOIN



Definition

A LEFT JOIN returns **all rows from the left table** (A), together with the matching rows from the right table (B). Where there is no match in B, the columns that come from B are filled with NULL. The word OUTER is optional: LEFT JOIN and LEFT OUTER JOIN are identical.

How it works

The left table is the one that is 'preserved'. Every row of A appears in the result at least once. If a row of A matches several rows of B, it appears once per match. If it matches none, it appears once with NULLs in all of B's columns.

When to use it

Use it when you need every record from the main table whether or not related data exists, for example, a list of all customers including those who have never ordered. It is also the standard way to find 'missing' relationships: filter for rows where a column from B is NULL to find A-rows without a match (an *anti-join* pattern).

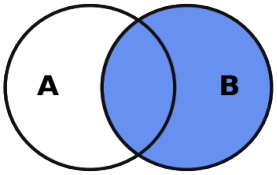
Handling of unmatched rows and NULLs

Unmatched rows from A are kept and padded with NULLs. Unmatched rows from B (order 104) are discarded.

Result on the sample data (4 rows):

CustomerID	Name	OrderID	Amount
1	Asha	101	500
1	Asha	102	300
2	Bala	103	700
3	Charu	NULL	NULL

5. RIGHT (OUTER) JOIN



Definition

A RIGHT JOIN returns **all rows from the right table** (B), together with the matching rows from the left table (A). Where there is no match in A, the columns that come from A are filled with NULL. RIGHT JOIN and RIGHT OUTER JOIN are identical.

How it works

It is the mirror image of the LEFT JOIN: now the right table is preserved. Every row of B appears at least once, and rows of A that have no partner are discarded.

When to use it

Use it when the table you must preserve happens to be written on the right side of the join, for example, listing all orders and the customer for each, including orders with no valid customer. In practice, most developers rewrite a RIGHT JOIN as a LEFT JOIN by swapping the order of the tables, because reading from left to right is easier to follow. The two forms give equivalent results.

Handling of unmatched rows and NULLs

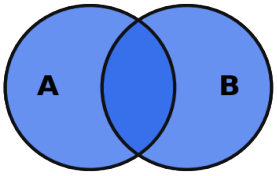
Unmatched rows from B are kept and padded with NULLs on the A side (order 104 appears with NULL customer details). Unmatched rows from A (customer 3) are discarded.

> **Key idea:** A LEFT JOIN B returns the same rows as B RIGHT JOIN A. Only the column order may differ.

Result on the sample data (4 rows):

CustomerID	Name	OrderID	Amount
1	Asha	101	500
1	Asha	102	300
2	Bala	103	700
NULL	NULL	104	200

6. FULL OUTER JOIN



Definition

A FULL OUTER JOIN returns **all rows from both tables**. Rows that match are combined; rows that exist in only one table are still included, with NULLs in the columns of the other table. It can be thought of as the union of the LEFT JOIN and the RIGHT JOIN results, with duplicates of matched rows removed.

How it works

The database first pairs up all rows that satisfy the join condition, then adds the leftover rows from A (padded with NULLs for B's columns), and finally the leftover rows from B (padded with NULLs for A's columns). Nothing from either table is lost.

When to use it

Use it when you want a complete picture of both sides and need to see where they do and do not agree, such as reconciling two systems, comparing two lists, or auditing records that exist in one source but not the other. Filtering the result for rows where either side is NULL isolates the mismatches.

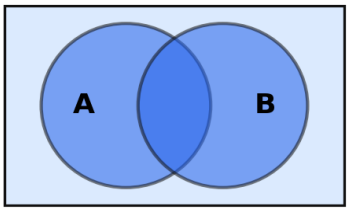
Handling of unmatched rows and NULLs

Unmatched rows from both sides are kept. Customer 3 appears with NULL order columns, and order 104 appears with NULL customer columns. Support for FULL OUTER JOIN varies between database systems; for example, MySQL does not provide it directly and it is usually simulated by combining a LEFT JOIN and a RIGHT JOIN.

Result on the sample data (5 rows):

CustomerID	Name	OrderID	Amount
1	Asha	101	500
1	Asha	102	300
2	Bala	103	700
3	Charu	NULL	NULL
NULL	NULL	104	200

7. CROSS JOIN



Definition

A CROSS JOIN returns the **Cartesian product** of the two tables: every row of A is paired with every row of B. It has **no join condition**, so no matching takes place at all.

How it works

If A has m rows and B has n rows, the result always has exactly **m x n** rows, and its columns are all columns of A followed by all columns of B. With our sample data, 3 customers and 4 orders give 12 rows. The result grows very quickly: two tables of 1,000 rows each would produce 1,000,000 rows.

When to use it

Use it deliberately when you need all possible combinations, for example, generating every combination of sizes and colours for a product catalogue, building a calendar by pairing dates with categories, or creating test data. Because the output size multiplies, it should be used with care on large tables.

Handling of unmatched rows and NULLs

Since there is no matching, there are no unmatched rows and no NULLs are introduced by the join. A frequent mistake is to forget the join condition in an ordinary join, which silently turns it into a Cartesian product with huge, incorrect results.

Key idea: an INNER JOIN can be understood as a CROSS JOIN followed by a filter that keeps only the pairs satisfying the join condition. The other join types are refinements of this idea.

8. Comparison of the Join Types

Join type	Rows returned	Unmatched rows from A	Unmatched rows from B	Needs condition?
INNER	Only matching rows	Dropped	Dropped	Yes
LEFT OUTER	All of A + matches from B	Kept (B columns NULL)	Dropped	Yes
RIGHT OUTER	All of B + matches from A	Dropped	Kept (A columns NULL)	Yes
FULL OUTER	All rows of A and B	Kept (B columns NULL)	Kept (A columns NULL)	Yes
CROSS	Every A row with every B row (m x n)	Not applicable	Not applicable	No

Quick way to remember

- **INNER**: only the overlap.
- **LEFT**: everything from the left, plus the overlap.
- **RIGHT**: everything from the right, plus the overlap.
- **FULL OUTER**: everything from both sides.
- **CROSS**: every possible pair, no matching.

Relationship between the types

INNER is a subset of LEFT, RIGHT and FULL: each of those returns all the inner-join rows plus extra unmatched rows. LEFT and RIGHT are in turn subsets of FULL. For the sample data, the row counts are INNER = 3, LEFT = 4, RIGHT = 4, FULL = 5 and CROSS = 12.

Row-count behaviour

- In a one-to-many join, the 'one' side is repeated once for each matching row on the 'many' side, so the result can have more rows than either input table.
- An INNER JOIN can never return more rows than the CROSS JOIN of the same tables, and never fewer rows than zero.
- LEFT JOIN always returns at least as many rows as there are in the left table; RIGHT JOIN at least as many as in the right table.
- Duplicate values in the join column on both sides multiply: if a key value appears 2 times in A and 3 times in B, that value alone produces 6 rows.

Common pitfalls

- **NULLs never match each other.** The comparison NULL = NULL is not true, so rows whose join column is NULL will not be matched by an ordinary equality join.
- **Missing join condition** turns a join into a Cartesian product and produces far more rows than expected.

- **Unintended duplicates** arise when the join column is not unique on the 'one' side, inflating totals such as sums and counts.
- **Filtering an outer join incorrectly.** A condition on the preserved-away table placed in the WHERE clause removes the NULL-padded rows and effectively converts an outer join into an inner join. Conditions meant to restrict the matching belong in the join condition itself.
- **Data type mismatch** between the joined columns can cause failed matches or slow queries.

Related concept: self join

A table can also be joined to *itself*, which is called a self join. It is not a separate join type but a technique that uses any of the types above with two aliases of the same table, typically to represent hierarchies such as employees and their managers.